Week 3 Code Review

Lessons Learned · AI Engineering Bootcamp · Compiled from instructor feedback

How to use this document

What follows is a distilled set of lessons drawn from this week's code review session. Each lesson combines a recurring instructor question with the principle behind it, so you are not just memorising answers but understanding *why* these things matter in production code. Treat the "Likely questions" under each lesson as a self-check: if you cannot answer them out loud, you do not yet own that part of your codebase.

> **The single biggest takeaway**
>
> You should be able to defend everything you write or that lives in your repository. If you copied a line, an import, a config flag, or a Docker directive without understanding it, expect to be asked about it. "I don't know" is fine once. Repeatedly is not.

1. Project Structure & Craftsmanship

1.1. Separate concerns into distinct folders and files

Your repository should make its architecture obvious at a glance. Keep **schemas**, **API routes**, **services**, and **utilities** in separate modules. Keep the **frontend** and **backend** in different top-level folders. Do not flood your project root with stray CSV files or notebooks.

When prompts grow, split them too. A single prompts.py with five different templates is a smell. Prefer priority_prompt.py, rag_prompt.py, and nonrag_prompt.py so each file has a single, obvious purpose.

Likely questions:

– Where would I find your API route definitions vs. your business logic?

– Why is this prompt living in the same file as your service code?

1.2. Keep files small and code simple

If a file has hundreds of lines, it is almost certainly doing too many things. Break it apart along the seams of responsibility. Verbose code is not impressive — it is harder to read, harder to review, and harder to test. Optimise for the next engineer (often: future you).

Watch out: If you cannot describe a file's job in one sentence, it needs to be split.

1.3. Use meaningful names everywhere

Functions, variables, files, classes, Docker services — all of them communicate intent through their names. process_data() tells me nothing. chunk_tweets_for_embedding() tells me everything. Naming is documentation.

1.4. Know your editor

Press **F12** (or your IDE's equivalent) to jump straight to a function definition. When the instructor asks to see a function, you should land on it in one keystroke — not scroll through files looking for it. Slow navigation tells the room you do not know your own codebase.

Likely questions:

– Show me the retrieve() function.

– In which file is the ingestion pipeline defined?

1.5. Use uv, not pip

uv is dramatically faster than pip, handles lockfiles cleanly, and is the direction the Python ecosystem is moving. Adopt it now and your environments will be reproducible and fast.

2. Walking Through Your Own Code

2.1. Be ready to give the tour — in two modes

You will be asked to demo your project in one of two ways:

- **Free-form:** "Walk me through the project." You set the order. Start from the entry point and follow the data through ingestion → storage → retrieval → response.
- **Instructor-led:** "Show me file X. Now file Y." You follow *their* order, even if it jumps around. Do not insist on your own narrative.

2.2. Explain ingestion and retrieval end-to-end

For a RAG project you must be able to narrate, in order: **load** → **clean** → **chunk** → **embed** → **store** on the ingest side, and **query** → **embed** → **similarity search** → **rerank (if any)** → **prompt assembly** → **generate** on the retrieval side. State exactly **how many results** you return at each step and **why** that number.

Likely questions:

– Walk me through the full ingestion and RAG pipeline.

– How many tickets / chunks are you returning from the retriever, and why?

– Did you use chunking, or did you treat each tweet as one chunk?

– What chunking technique did you use?

2.3. Know how often each function runs

How many times is your ingestion function called? Once at startup? On every request? On a schedule? Confusing "runs once" with "runs per request" is a very common bug — and a very common interview question.

3. Data, ML & Evaluation

3.1. Own your ML pipeline

You should be able to walk through every stage: feature engineering, train/test split, model training, validation, threshold calibration, and final metrics. Be specific about the **label** you are predicting and the **features** you chose and why.

Likely questions:

– What is the label you are predicting?

– Walk me through your feature engineering. How did you choose your features?

– Why did you use cross-validation?

– Why did you use stratify in train_test_split?

– Why did you calibrate the threshold?

Watch out: stratify is about preserving class balance across splits. If your classes are imbalanced and you forget it, your test set may not reflect reality.

3.2. Defend your metrics — do not just display them

Showing accuracy, precision, recall, and F1 is the *start* of the conversation, not the end. Interpret the numbers: which class is the model weakest on? What does a false positive cost in this domain vs. a false negative? If you cannot turn the metric into a sentence about real-world behaviour, the metric means nothing.

Likely questions:

– What are precision and recall?

– What does this F1 score actually tell you about the system?

3.3. Justify every preprocessing choice

Every transformation in your pipeline is a decision you must defend: vectoriser, stop words, embedding model, batching strategy. Stop words, for example, can hurt semantic search — so why are they on?

Likely questions:

– What vectoriser are you using and why?

– Why are you using stop words?

– What embedding model did you use — Hugging Face, Gemini, OpenAI? Why that one?

– Did you check whether your embedding model runs in batches? Is it efficient?

3.4. Know your dependencies — including the heavy ones

If you imported PyTorch or sentence-transformers, know *why*. Sentence-transformers depends on PyTorch under the hood; that is fine, but you should be able to say so and explain the trade-off (size, GPU support, latency) versus a hosted embedding API.

4. RAG Specifics

4.1. Understand your vector store

Chroma is a **filesystem-backed** database. That is fine for prototyping, but it is not the right choice for production — production systems use a real **server-based database** (Postgres + pgvector, Qdrant, Weaviate, Pinecone, etc.). Know where your Chroma data lives on disk and know what metadata you persist alongside the vectors.

Likely questions:

– *What type of database is Chroma? Where is the data saved?*

– *What metadata do you save alongside each vector?*

– *Where on disk is your Chroma data?*

Watch out: Filesystem DB ≠ in-memory DB ≠ server DB. Know which you are using and why.

4.2. grounded_answer vs. rag_answer

If you have two answer-generation paths, the difference must be deliberate and explainable. Typically: one is grounded strictly in retrieved context (refuses to answer if context is missing); the other falls back to the LLM's parametric knowledge. Be ready to say which is which and when each runs.

4.3. Tool orchestration: parallel vs. sequential

If your system has multiple tools (RAG, raw LLM, ML classifier), be explicit about how you call them. Are they run in parallel and compared? Are they routed to based on a classifier? Are they fallbacks for each other? There is no single right answer — but there is a wrong answer, which is "I'm not sure."

5. Python Craft

5.1. Cache your clients with lru_cache

Do not initialise the OpenAI client (or any external client) on every call. Wrap the constructor in @functools.lru_cache so it is built once and reused. While you are there, learn what **cache invalidation** means and when an lru_cache would be the wrong choice (e.g. when the underlying config can change at runtime).

Likely questions:

– Why are you initialising the client every time?

– Why did you use lru_cache here? What is cache invalidation?

5.2. Async, threads, and where the bottleneck is

Async matters because it keeps the user interface responsive — a blocking call in an async endpoint freezes the whole event loop. Know the difference between a **CPU-bound** task (use a process pool or a worker) and an **I/O-bound / network-bound** task (use async or threads). asyncio.to_thread is the right tool for offloading a blocking call from inside async code.

Likely questions:

– What is asyncio.to_thread?

– How does using async vs. sync affect the user experience?

– What is the difference between CPU-bound and network-bound work? How do you handle each?

5.3. OOP fundamentals you will be quizzed on

These are not trick questions — they are the absolute basics. Know them cold:

- **__init__**: the constructor; runs when the class is instantiated and sets up instance state.
- **@staticmethod**: a method that does not need self and can be called without instantiating the class.
- **@property**: turns a method into an attribute-style access — read-only by default, useful for computed values.
- **Dependency injection**: passing dependencies into a class instead of constructing them inside it. Makes code testable. Research it before next session.

6. Docker & Deployment

6.1. Defend every line of your Dockerfile and compose file

Be ready to walk through your Dockerfiles and explain how the containers talk to each other. If you used a directive, you must know what it does — "I copied it from somewhere" is not an answer.

6.2. Networks vs. volumes — do not confuse them

Networks let containers reach each other by service name. **Volumes** persist data so it survives container restarts and removal. These solve completely different problems. A common mistake in this review was claiming networks provide persistence — they do not. Volumes do.

Likely questions:

– Why did you use networks in docker-compose.yml?

– Why did you use volumes?

Watch out: If the container is destroyed, network membership is gone — but volume data survives.

6.3. Never hardcode environment-specific URLs

Your React URL, your database URL, your model endpoint — none of these should be baked into docker-compose.yml. Use environment variables with sensible fallbacks, e.g. ${REACT_URL:-http://localhost:3000}.

6.4. CORS: know what it does and why you need it

CORS (Cross-Origin Resource Sharing) is the browser's mechanism for allowing or blocking requests from one origin to another. add_middleware(CORSMiddleware, …) in FastAPI tells the server which origins, methods, and headers to permit. Get it wrong and your frontend cannot talk to your backend at all.

Likely questions:

– What does cors_origins do?

– Why are you using add_middleware with CORS middleware?

7. Observability & Production Readiness

7.1. Logging is not print()

A real logging_setup.py configures levels, formatters, and handlers — and writes to a **persistent file**, not just the terminal. Logs that only exist in stdout are gone the moment the container restarts.

Likely questions:

– Walk me through your logging_setup.py.

– Where are logs being written — to the terminal, or to a file?

7.2. App-to-app authentication exists for a reason

In production, services do not call each other anonymously. They authenticate using **OAuth2 client credentials**, **mTLS**, or service-account tokens. Research app-to-app auth before your next project — it is the difference between a school demo and something that could ship.

7.3. Bring forward what you learned last week

Each week's material is cumulative. If something from week 1 or 2 fits — guardrails, structured outputs, prompt patterns, evals — use it. Code reviewers notice when you don't.

The meta-rule for every code review

Every choice in your repository is a question waiting to be asked. Before your next review, walk through your own code as if you were the instructor. If a line, a flag, a library, or a folder structure makes you hesitate — that is exactly the line you will be asked about. Fix the gap before the session, not during it.
